UNIVERSIDAD POLITÉCNICA DE ATLAUTLA

Ingeniería en Tecnologías de la Información e Innovación Digital

# ACTIVIDAD 3

# Búsqueda local y optimización

| Datos de entrega | Información |
|---|---|
| Asignatura | Fundamentos de Inteligencia Artificial |
| Alumno | Emmanuel Guzmán García |
| Grupo | 172 |
| Docente | Mtro. Javier García Lara |
| Fecha indicada en el enunciado | 22 de octubre de 2026 |

Repositorio: https://github.com/xbae23/FIA_Parcial2_Emmanuel_Guzman

Demostración: https://xbae23.github.io/FIA_Parcial2_Emmanuel_Guzman/

Portafolio individual versionado en GitHub. Se incluyen planteamiento, procedimiento, evidencia, conclusiones y referencias.

## Enunciado y función objetivo

Minimizar una función real de dos variables sobre $[-5.12, 5.12]^2$. Se usa Rastrigin como paisaje de prueba multimodal: **$f(x,y) = 20 + x^2 + y^2 - 10[\cos(2\pi x) + \cos(2\pi y)]$**. Su mínimo global conocido es $f(0,0)=0$. Es un problema matemático didáctico, no un caso empresarial real. La vecindad de Hill Climbing son los cuatro puntos a distancia 0.1 por eje.

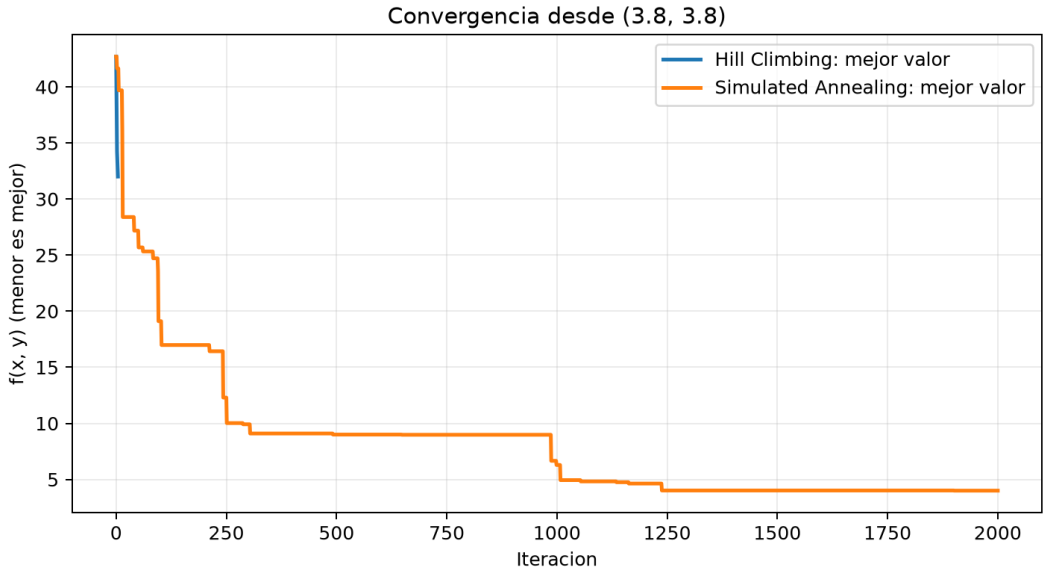
## Métodos y parámetros

| Método | Regla | Parámetros |
|---|---|---|
| Hill Climbing | Elige el mejor vecino si mejora; termina si no mejora. | Paso 0.1; máximo 500 iteraciones. |
| Simulated Annealing | Acepta empeoramiento con probabilidad $\exp(-\Delta/T)$; registra el mejor visto. | T inicial 10; $T \leftarrow 0.995T$; 2000 iteraciones; ruido gaussiano $\sigma=0.25$. |

Ambos métodos aplican límites al dominio. El experimento de convergencia comienza en (3.8, 3.8), con semilla 42 para recocido simulado. La comparación adicional usa 30 puntos iniciales generados con semillas 0–29; el recocido usa la misma semilla de cada corrida.

## Resultados reproducibles

| Medida | Hill Climbing | Simulated Annealing |
|---|---|---|
| Inicio (3.8, 3.8): f final | 32.0000 | 4.0013 |
| Iteraciones del ejemplo | 4 | 2000 |
| Media de f final, 30 inicios | 18.4236 | 7.0710 |
| Corridas con f < 1, de 30 | 0 | 2 |




La gráfica registra el mejor valor encontrado, no el estado instantáneo del recocido. Por eso ambas curvas son no crecientes aunque el recocido acepte pasos temporales de mayor costo.

## Código, verificación y complejidad

**Act3/act3_optimizacion.py** incluye `objetivo`, `hill_climbing`, `simulated_annealing` y la exportación de resultados. Cada evaluación de f es O(1); HC usa hasta O(4I) evaluaciones con memoria O(I) para el historial, y SA usa O(I) tiempo y O(I) memoria por su historial. Sin guardar gráficas, la memoria de optimización sería O(1). Las pruebas comprueban f(0,0)=0 y que ambos métodos conservan una solución no peor que la inicial.

## Conclusiones y límites

En esta configuración el recocido obtuvo menor valor medio, pero no alcanzó el mínimo global en la mayoría de corridas. Hill Climbing quedó atrapado cerca de mínimos locales por la vecindad fija. El resultado depende de parámetros, presupuesto de evaluaciones y semilla; 30 corridas son una comparación exploratoria, no una garantía general.

## Referencias (APA 7.ª ed.)

Russell, S., & Norvig, P. (2020). Artificial intelligence: A modern approach (4th ed.). Pearson.

Wirsansky, E. (2020). Hands-on genetic algorithms with Python. Packt. [Referencia del temario; no se implementan algoritmos genéticos en esta actividad].

## Uso de inteligencia artificial y verificación

Se utilizó ChatGPT/Codex para apoyar la organización del reporte, la redacción, el desarrollo de código y gráficos. Se ejecutaron los programas y pruebas locales, y se cotejaron las afirmaciones documentales con las fuentes citadas. Quienes presentan el trabajo deben revisarlo, comprenderlo y poder explicarlo.

## Anexo: código Python completo

Archivo ejecutable **Act3/act3_optimizacion.py**. La numeración facilita la revisión; el archivo del repositorio es la versión directamente ejecutable.

```
001  """Minimización reproducible de Rastrigin en dos dimensiones."""
002  from __future__ import annotations
003
004  import json
005  import math
006  import random
007  from pathlib import Path
008
009  import matplotlib
010  matplotlib.use("Agg")
011  import matplotlib.pyplot as plt
012
013  BASE = Path(__file__).resolve().parent
014  OUT = BASE / "resultados"
015  LIMITE = 5.12
016
017
018  def objetivo(x: tuple[float, float]) -> float:
019      """Rastrigin: f(x,y)=20+x²+y²-10(cos(2πx)+cos(2πy)); mínimo 0."""
020      return 20 + sum(v * v - 10 * math.cos(2 * math.pi * v) for v in x)
021
022
023  def limitar(x: tuple[float, float]) -> tuple[float, float]:
024      return tuple(max(-LIMITE, min(LIMITE, v)) for v in x)
025
026
027  def hill_climbing(inicio: tuple[float, float], paso: float = 0.1,
028                    max_iter: int = 500) -> tuple[tuple[float, float], list[float]]:
029      """Descenso de mejor vecino en malla; termina al no mejorar."""
030      actual = limitar(inicio)
031      historial = [objetivo(actual)]
032      for _ in range(max_iter):
033          vecinos = [limitar((actual[0] + dx, actual[1] + dy))
034                     for dx, dy in ((paso, 0), (-paso, 0), (0, paso), (0, -paso))]
035          mejor = min(vecinos, key=objetivo)
036          if objetivo(mejor) >= objetivo(actual) - 1e-12:
037              break
038          actual = mejor
039          historial.append(objetivo(actual))
040      return actual, historial
041
042
043  def simulated_annealing(inicio: tuple[float, float], semilla: int,
044                          t0: float = 10.0, enfriamiento: float = 0.995,
045                          iteraciones: int = 2000, sigma: float = 0.25
046                          ) -> tuple[tuple[float, float], list[float]]:
047      """Acepta empeoramientos con probabilidad exp(-delta/T); conserva el mejor."""
048      rng = random.Random(semilla)
049      actual = limitar(inicio)
050      mejor = actual
051      historial = [objetivo(mejor)]
052      for i in range(iteraciones):
053          temperatura = t0 * enfriamiento ** i
054          candidato = limitar((actual[0] + rng.gauss(0, sigma),
055                               actual[1] + rng.gauss(0, sigma)))
056          delta = objetivo(candidato) - objetivo(actual)
057          if delta <= 0 or rng.random() < math.exp(-delta / temperatura):
058              actual = candidato
059          if objetivo(actual) < objetivo(mejor):
060              mejor = actual
061          historial.append(objetivo(mejor))
062      return mejor, historial
063
064
065  def ejecutar() -> dict:
066      OUT.mkdir(exist_ok=True)
067      inicio = (3.8, 3.8)
068      hc_x, hc_hist = hill_climbing(inicio)
069      sa_x, sa_hist = simulated_annealing(inicio, semilla=42)
070      semillas = range(30)
071      valores_hc = []
072      valores_sa = []
073      for semilla in semillas:
074          rng = random.Random(semilla)
075          punto = (rng.uniform(-LIMITE, LIMITE), rng.uniform(-LIMITE, LIMITE))
076          valores_hc.append(objetivo(hill_climbing(punto)[0]))
077          valores_sa.append(objetivo(simulated_annealing(punto, semilla)[0]))
078      fig, ax = plt.subplots(figsize=(8, 4.5))
079      ax.plot(range(len(hc_hist)), hc_hist, label="Hill Climbing: mejor valor", linewidth=2)
080      ax.plot(range(len(sa_hist)), sa_hist, label="Simulated Annealing: mejor valor", linewidth=2)
```

```
    ax.set(xlabel="Iteracion", ylabel="f(x, y) (menor es mejor)", title="Convergencia desde (3.8, 3.8)")
    ax.legend()
    ax.grid(alpha=0.25)
    fig.tight_layout()
    fig.savefig(OUT / "act3_convergencia.png", dpi=180)
    plt.close(fig)
    datos = {
        "funcion": "Rastrigin 2D", "dominio": [-LIMITE, LIMITE],
        "inicio_comun": inicio,
        "parametros": {"hc_paso": 0.1, "hc_max_iter": 500,
                       "sa_t0": 10.0, "sa_enfriamiento": 0.995,
                       "sa_iteraciones": 2000, "sa_sigma": 0.25, "sa_semilla_ejemplo": 42},
        "ejemplo": {"hill_climbing": {"x": hc_x, "valor": objetivo(hc_x), "iteraciones": len(hc_hist)-1},
                    "simulated_annealing": {"x": sa_x, "valor": objetivo(sa_x), "iteraciones": len(sa_hist)-1}},
        "30_inicios": {"media_hc": sum(valores_hc)/30, "media_sa": sum(valores_sa)/30,
                       "mediana_hc": sorted(valores_hc)[14:16],
                       "mediana_sa": sorted(valores_sa)[14:16],
                       "exitos_hc_f_menor_1": sum(v < 1 for v in valores_hc),
                       "exitos_sa_f_menor_1": sum(v < 1 for v in valores_sa),
                       "valores_hc": valores_hc, "valores_sa": valores_sa},
    }
    (OUT / "act3_resultados.json").write_text(json.dumps(datos, indent=2, ensure_ascii=False), encoding="utf-8")
    print(json.dumps(datos, indent=2, ensure_ascii=False))
    return datos


if __name__ == "__main__":
    ejecutar()

```